



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-G2

System Integration and Architecture

System Design Document (SDD)

Project Title: DentaLink – Dental Appointment System

Prepared By: Riconalla, Frances Anne Briones

Revision History Table

Version	Date	Author	Changes Made	Status
1	02/23/2026	Frances Anne B. Riconalla	Initial draft (Executive Summary, FRS, and NFR)	Draft
2	03/01/2026	Frances Anne B. Riconalla	Added UI mockups, diagrams, endpoint specifications, project timeline	Final
3	03/08/2026	Frances Anne B. Riconalla	Minor changes in endpoints	Revised

Table of Contents

Revision History Table	2
Table of Contents.....	3
Executive Summary	5
1.1 Project Overview	5
1.2 Objectives.....	5
1.3 Scope	5
1.0 Introduction	7
1.1 Purpose	7
2.0 Functional Requirements Specification	7
2.1 Project Overview	7
2.2 Core User Journeys	7
2.3 Feature List (MoSCoW)	8
2.4 Detailed Feature Specifications	9
2.5 Acceptance Criteria	12
3.0 Non-Functional Requirements.....	13
3.1 Performance Requirements	13
3.2 Security Requirements.....	14
3.3 Reliability Requirements	14
3.4 Availability Requirements.....	14
3.5 Compatibility Requirements	15
3.6 Usability Requirements	15
4.0 System Architecture.....	15
4.1 Architecture Pattern	15
4.2 Technology Stack.....	15
4.3 Component Diagram	17
5.0 API Contract & Communication.....	18
5.1 API Standards	18
5.2 HTTP Status Codes	19
5.3 Endpoint Specifications.....	19
6.0 Database Design	27
6.1 Entity Relationship Diagram.....	27
7.0 UI/UX Design	29
7.1 Web Application Wireframes	29
7.2 Mobile Application Wireframes	38
8.0 Plan	47

8.1 Project Timeline 47

8.2 Milestones 47

8.3 Risk Mitigation 47

8.4 Repository Structure 48

Executive Summary

1.1 Project Overview

DentaLink is a centralized appointment and payment management system designed for a single dental clinic. It enables patients to securely register, log in, browse dental services, book appointments, and pay for those appointments using PayMongo (Sandbox Mode). The system provides clinic administrators with a role-based management dashboard to oversee services, dentists, and appointments.

The system integrates the following components:

- Spring Boot backend API (Java 17+, Spring Security, JWT, JPA)
- React web application consuming the backend REST API
- Android mobile application (Kotlin, XML Layouts, Retrofit, API Level 34)
- PostgreSQL database (hosted on Supabase Free Tier)
- PayMongo payment gateway (Sandbox Mode)
- Google OAuth 2.0 for social login
- Google Maps JavaScript API for clinic location display
- SMTP email service for appointment confirmations
- File upload for service images and patient profile pictures (stored in Supabase Storage)

1.2 Objectives

1. Develop a secure multi-tier dental appointment and payment system
2. Implement JWT authentication with Google OAuth support
3. Enforce role-based access control for Admin and Patient roles
4. Integrate PayMongo (Sandbox) for online payment processing
5. Implement file upload for service images and patient profile photos
6. Integrate Google Maps API to display clinic location
7. Implement SMTP email notifications for appointment confirmations
8. Provide a fully functional Android mobile application connected to the same backend
9. Deploy backend on Render and web frontend on Vercel

1.3 Scope

Included Features:

- User registration (patient only) and login with email/password and Google OAuth
- JWT-based authentication with BCrypt password hashing
- Role-based access control: ADMIN and PATIENT
- Dental service management — full CRUD with image upload (admin)
- Dentist record management — full CRUD (admin, no login for dentists)
- Appointment booking with double-booking prevention and clinic operating-hour validation
- Patient cancellation of unpaid (PENDING_PAYMENT) appointments before payment

- Automatic cancellation of unpaid appointments left unpaid for more than 60 minutes
- PayMongo payment integration (Sandbox) — auto-confirmation after payment via webhook
- SMTP email notification on registration (welcome) and appointment confirmation
- Google Maps API integration for clinic location display
- Patient profile management with profile picture upload
- Admin dashboard with appointment and payment monitoring
- Android mobile app with JWT auth and protected API access (patient and admin functions)

Excluded Features:

- Real multi-clinic (SaaS) support
- Dentist login accounts (dentists are internal records only)
- Patient-initiated cancellation of paid (CONFIRMED) appointments
- Refund processing
- Dentist reassignment after booking
- Automatic rescheduling
- Real-time WebSocket features
- SMS notifications
- Insurance claim processing
- Production (live) payment processing
- Advanced analytics dashboard

1.0 Introduction

1.1 Purpose

This document serves as the comprehensive design specification for DentaLink — a dental appointment and payment management system. It provides detailed functional requirements, architectural decisions, full API contracts, database schema, system integrations, and an implementation roadmap to guide development across the backend, web frontend, and Android mobile application.

2.0 Functional Requirements Specification

2.1 Project Overview

Project Name	DentaLink — Dental Appointment & Payment Management System
Domain	Healthcare / Dental Clinic Services
User Roles	ADMIN (pre-created), PATIENT (self-register)
Backend	Spring Boot (Java 17+), JWT, BCrypt, JPA — Layered Architecture
Web Frontend	React — consumes Spring Boot REST API
Mobile App	Android Kotlin + XML Layouts + Retrofit (API Level 34)
Database	PostgreSQL (Supabase Free Tier)
Integrations	PayMongo, Google OAuth 2.0, Google Maps API, SMTP Email, Supabase Storage

Problem Statement: Manual appointment booking in dental clinics leads to scheduling conflicts, inefficient payment handling, and limited digital record management for both patients and staff.

Solution: DentaLink provides a secure and integrated dental appointment booking and payment system accessible via web and mobile platforms, with streamlined workflows for both patients and administrators.

2.2 Core User Journeys

Journey 1: Patient Registration and Login

1. Patient visits the web or mobile application
2. Registers via email/password or Google OAuth
3. Password stored using BCrypt hashing
4. A welcome email is sent to the patient upon successful registration
5. JWT issued upon successful login
6. Patient accesses protected routes based on PATIENT role

7. GET /auth/me returns currently authenticated user information

Journey 2: Patient Books an Appointment

1. Patient logs in and navigates to the Services page
2. Views available dental services with images, descriptions, and prices
3. Clicks 'Book Now' on a service
4. Selects preferred dentist (only ACTIVE dentists are listed to patients), date, and time slot
5. System validates the request: the date/time must be in the future, must not fall on a Sunday (clinic closed), must fall within operating hours (Mon–Fri 8:00 AM–5:00 PM, Sat 9:00 AM–5:00 PM), and the selected dentist must be ACTIVE
6. System validates that the selected dentist is not already booked at that date/time
7. If any validation fails — the system returns a 400 Bad Request (business-rule violation) or a 409 Conflict (double-booking) and the patient must adjust the selection
8. Appointment created with status = PENDING_PAYMENT and payment_status = UNPAID
9. Patient redirected to the PayMongo hosted checkout page
10. If payment is not completed within 60 minutes, a background scheduler automatically cancels the appointment (status = CANCELLED) and releases the slot for rebooking
11. On payment success — PayMongo webhook received, appointment auto-updated to status = CONFIRMED, payment_status = PAID
12. Confirmation email sent to patient via SMTP
13. Patient can view confirmed appointment in 'My Appointments'
14. Patient may cancel an appointment only while it is still unpaid (PENDING_PAYMENT); cancellation is not permitted after payment is completed

Journey 3: Admin Manages the System

1. Admin logs in using pre-created credentials (no public registration for admin)
2. Admin dashboard displays summary statistics: total appointments, pending payments, confirmed appointments, completed/cancelled/pending-payment counts, and revenue
3. Admin navigates to Manage Services — can create, edit, delete services and upload service images
4. Admin navigates to Manage Dentists — can add, edit, delete dentist records
5. Admin views all appointments — can update status to CANCELLED, or to COMPLETED only when the appointment has been paid (payment_status = PAID)
6. Admin cannot reassign dentists after booking
7. Admin views payment records and PayMongo payment IDs
8. Admin cannot fake or manually trigger payment confirmations (the CONFIRMED status is set only by the verified PayMongo webhook)

2.3 Feature List (MoSCoW)

MUST HAVE

1. User registration (PATIENT only) and login with email/password

2. Google OAuth login with JWT generation
3. JWT authentication with BCrypt password hashing
4. Protected routes and /auth/me endpoint
5. Role-based access control: ADMIN and PATIENT (API-level + UI-level)
6. Dental service management — full CRUD with image upload (Admin)
7. Dentist record management — full CRUD (Admin; no dentist login)
8. Appointment booking — patient selects service, dentist, date, and time
9. Double-booking prevention per dentist at same date/time (409 Conflict)
10. Booking validation — future date, clinic operating hours, no Sunday, active dentist
11. Appointment status: PENDING_PAYMENT → CONFIRMED → COMPLETED / CANCELLED
12. PayMongo Sandbox payment integration — webhook-based auto-confirmation
13. Automatic cancellation of unpaid appointments after a 60-minute timeout
14. SMTP email notification on registration (welcome) and appointment confirmation
15. Google Maps API integration — clinic location display
16. Admin dashboard with statistics and recent appointments
17. File upload for service images (Admin) and profile pictures (Patient)

SHOULD HAVE

1. Appointment filtering by status in admin view
2. Basic form validation feedback on frontend
3. Responsive UI for desktop and mobile web
4. Patient appointment history view
5. Patient self-cancellation of unpaid appointments

COULD HAVE

1. Revenue summary and appointment-status chart on admin dashboard
2. Search/filter services by name
3. Patient profile editing beyond profile picture

WON'T HAVE

1. Patient cancellation after payment / refund processing
2. Dentist login accounts
3. Dentist reassignment after booking
4. SMS notifications
5. Real-time WebSocket features
6. Multi-branch clinic support
7. Production (live) payment processing

2.4 Detailed Feature Specifications

Feature: User Authentication

- Screens: Registration Page, Login Page
- Fields: Email, Password, Confirm Password, First Name, Last Name
- Validation: Email format, password strength (minimum 8 characters), unique email constraint
- API Endpoints: POST /auth/register, POST /auth/login, POST /auth/logout, GET /auth/me, POST /auth/google
- Security: JWT tokens, BCrypt password hashing, role-based route protection, DTOs to exclude password from responses

Feature: Service Management (Admin)

- Screens: Admin Manage Services page — Service list with Create / Edit / Delete actions
- Display: Card/table with service name, description, price, image, actions
- Admin Functions: Create service, update service details, delete service, upload service image
- API Endpoints: GET /services, GET /services/{id}, POST /services, PUT /services/{id}, DELETE /services/{id}, POST /services/{id}/upload-image
- Access Control: Create, update, delete, and upload require ADMIN role. GET /services and GET /services/{id} are public.
- Note: A service that is already referenced by existing appointments cannot be deleted (returns 400).

Feature: Dentist Management (Admin)

- Screens: Admin Manage Dentists page — Dentist list with Add / Edit / Delete actions
- Fields: Name, Specialization, Status (Active/Inactive)
- Note: Dentists are internal records only. They do not have login accounts.
- API Endpoints: GET /dentists, POST /dentists, GET /dentists/{id}, PUT /dentists/{id}, DELETE /dentists/{id}
- Access Control: All modification endpoints require ADMIN role. GET /dentists returns all dentists to ADMIN but only ACTIVE dentists to PATIENT.
- Note: A dentist already referenced by existing appointments cannot be deleted (returns 400); set their status to INACTIVE instead.

Feature: Appointment Booking (Patient)

- Screens: Services page with 'Book Now' button, Booking form/modal (select dentist, date, time), Redirect to PayMongo
- Process: Patient selects service → selects dentist → selects date and time → system validates business rules and double-booking → appointment created → redirected to PayMongo
- API Endpoints: POST /appointments, GET /appointments, GET /appointments/{id}, POST /appointments/{id}/cancel
- Business Rules: (1) datetime must be in the future; (2) Sundays are not allowed (clinic closed); (3) time must fall within operating hours — Mon–Fri 8:00 AM–5:00 PM, Sat 9:00 AM–5:00 PM; (4) the dentist must be ACTIVE; (5) the dentist must not already have a

non-cancelled appointment at the same date/time. A double-booking returns 409 Conflict; the other rule violations return 400 Bad Request.

- Status on creation: status = PENDING_PAYMENT, payment_status = UNPAID
- Cancellation: A patient may cancel their own appointment only while payment_status = UNPAID, via POST /appointments/{id}/cancel.

Appointment Status Values

Status Value	Meaning / Trigger
PENDING_PAYMENT	Appointment created. Payment not yet completed. Set automatically on booking.
CONFIRMED	Payment verified via PayMongo webhook. Appointment is confirmed. Auto-set — this status can only be set by the verified webhook, never manually.
COMPLETED	Admin marks the appointment as completed after the dental visit. Allowed only when payment_status = PAID; an unpaid appointment cannot be marked COMPLETED.
CANCELLED	Set by any of three triggers: (1) Admin cancels the appointment (internal reason); (2) Patient cancels their own still-unpaid appointment; (3) the background scheduler auto-cancels an appointment left PENDING_PAYMENT/UNPAID for more than 60 minutes. Patients cannot cancel after payment.

Feature: Payment Integration (PayMongo Sandbox)

- Screens: Payment redirect to the PayMongo hosted checkout page
- Process: Backend creates a PayMongo Checkout Session → returns the hosted checkout URL → patient redirected → on success: webhook received → appointment status updated to CONFIRMED, payment_status = PAID → confirmation email sent
- API Endpoints: POST /payments/create-intent (creates the Checkout Session), POST /payments/webhook (webhook from PayMongo), GET /payments (admin monitoring)
- Mode: Sandbox only. No live transactions.
- Restriction: Admin cannot manually confirm payment. No refund processing.
- Note: Although the create-intent response field is named paymentIntentId, the integration uses a PayMongo Checkout Session and the value returned is the checkout session ID (cs_...). This session ID is stored on the appointment and used to correlate the incoming webhook.

Payment Status Values

Payment Status Value	Meaning
UNPAID	No payment received. Initial state on booking.
PAID	PayMongo webhook confirmed successful payment.

Feature: Email Notification (SMTP)

- Triggers: (1) Welcome email sent immediately upon successful patient registration; (2) Appointment confirmation email sent automatically after the PayMongo webhook confirms successful payment
- Integration: SMTP server configuration (JavaMailSender) using secure credentials stored in environment variables. Console print is NOT accepted as a substitution.

Feature: Google Maps API Integration

- Purpose: Display the clinic's physical location for navigation assistance
- Functionality: Interactive map embedded on a Contact/Location page. Marker displays the clinic name at fixed coordinates. Users can open navigation directions via Google Maps.
- Web: Google Maps JavaScript API embedded in the React frontend
- Mobile: An embedded Google Maps view (WebView) plus an external Google Maps geo Intent to open the device navigation app

Feature: File Upload

- Service Image (Admin): Admin uploads a JPEG/PNG image when creating or editing a service. The file is uploaded to Supabase Storage and the resulting public URL is saved in the database. The image is displayed on the Services page.
- Profile Picture (Patient): Patient uploads a JPEG/PNG profile photo from the Profile page. The file is uploaded to Supabase Storage and the public URL is saved in the users table. It is displayed on the patient profile.
- Restrictions: File type limited to JPEG/PNG. File size validated at a maximum of 5 MB.

Feature: Admin Dashboard

- Display: Total Appointments, Pending Payments count, Confirmed Appointments count, Completed/Cancelled/Pending-Payment counts, Revenue Summary, and a Recent Appointments table
- Navigation: Dashboard | Manage Services | Manage Dentists | Manage Appointments | Payments | Logout

Feature: Patient Dashboard

- Display: Welcome message, summary stat cards (Upcoming, Completed, Cancelled / Pending counts), an upcoming appointment card, and a 'Book Appointment' / Quick Actions section
- Navigation: Dashboard | Services | My Appointments | Profile | Contact | Logout
- My Appointments: View all own appointments with service, dentist, date, status, and payment status. Cannot cancel after payment.

2.5 Acceptance Criteria

AC-1: Successful User Registration

Given I am a new user

When I enter a valid email, password, first name, and last name and submit the registration form

Then my account is created with role = PATIENT and password hashed using BCrypt

And a welcome email is sent to my registered email address

And I can log in using my credentials

AC-2: Google OAuth Login

Given I click 'Login with Google'

When I authorize the application

Then my Google account is created (or matched by email) in the database

And a JWT is issued for subsequent authenticated requests

AC-3: Admin Creates a Service

Given I am logged in as Admin

When I enter service details and upload a valid image

Then the service appears in the service listing visible to patients

AC-4: Appointment Booking and Auto-Confirmation

Given I am a logged-in Patient

When I select a service, dentist, and available time slot within clinic operating hours

And complete payment successfully via PayMongo

Then the appointment status is automatically updated to CONFIRMED

And a confirmation email is sent to my registered email address

And the appointment appears in My Appointments

AC-5: Double-Booking Prevention

Given a dentist already has a non-cancelled appointment at a specific date and time

When another patient attempts to book the same dentist at the same date and time

Then the system returns a 409 Conflict error

And an appropriate error message is displayed to the patient

AC-6: Post-Payment Cancellation Restriction

Given a patient has a CONFIRMED appointment with payment_status = PAID

When the patient attempts to cancel the appointment

Then the system does not allow the cancellation

And an appropriate message is displayed

AC-7: Unpaid Appointment Auto-Expiry

Given a patient has a PENDING_PAYMENT appointment that has remained UNPAID for more than 60 minutes

When the background scheduler runs

Then the appointment status is set to CANCELLED

And the time slot becomes available for rebooking

AC-8: Google Maps Display

Given I access the Contact / Location page

When the page loads

Then an interactive map shows the clinic location with a marker

And I can click to open navigation directions

3.0 Non-Functional Requirements

3.1 Performance Requirements

- API response time ≤ 2 seconds for 95% of requests under normal load
- Web page load time ≤ 3 seconds on a stable broadband connection
- Mobile application cold start time ≤ 3 seconds
- System supports at least 100 concurrent users
- Database queries complete within 500 milliseconds for standard operations
- Appointment slot validation executes immediately before booking confirmation

3.2 Security Requirements

- All API communications must use HTTPS
- JWT authentication required for all protected endpoints
- Passwords hashed using BCrypt with secure salt rounds
- Role-based access control enforced at both API (controller @PreAuthorize) and UI levels
- File upload restricted to JPEG and PNG formats with a 5 MB file size limit
- PayMongo, Google Maps, Google OAuth, Supabase, and SMTP credentials stored securely in environment variables
- DTOs used in all API responses — password field never exposed
- SQL injection prevention via JPA parameterized queries
- Admin and patient endpoints are role-restricted at the controller level
- PayMongo webhook signature verification (HMAC-SHA256) required before processing
- Cross-Origin Resource Sharing (CORS) is configured to allow API access only from trusted frontend origins such as the deployed web application.

3.3 Reliability Requirements

- System prevents duplicate bookings for the same dentist at the same date and time (enforced at the application/service layer; cancelled slots are released for rebooking)
- Payment status cannot be manually modified by any user
- Appointment confirmation only occurs after a verified PayMongo webhook
- Failed payment must not update appointment status to CONFIRMED
- A background scheduler runs every 5 minutes and auto-cancels appointments left PENDING_PAYMENT/UNPAID for more than 60 minutes, releasing the slot for rebooking
- System maintains data consistency during concurrent booking attempts
- Database backup supported via Supabase managed PostgreSQL

3.4 Availability Requirements

- Backend deployed on Render (cloud environment)
- Database hosted on Supabase Free Tier (PostgreSQL)
- Web application publicly accessible via Vercel
- Target uptime: at least 95% under normal operating conditions
- Application handles temporary connection interruptions gracefully

3.5 Compatibility Requirements

- Web application: Chrome, Firefox, Safari, Edge (latest two versions)
- Android mobile application: API Level 34 (Android 14) — strict requirement
- Operating systems: Windows 10+, macOS 10.15+, Linux browsers

3.6 Usability Requirements

- Booking process completable within a single page flow or modal
- Clear validation messages for all input fields
- Consistent navigation pattern across web and mobile platforms
- Touch-optimized buttons (minimum 44x44px) on mobile
- Accessible design with readable typography and sufficient color contrast
- Error messages provide clear recovery guidance to the user

4.0 System Architecture

4.1 Architecture Pattern

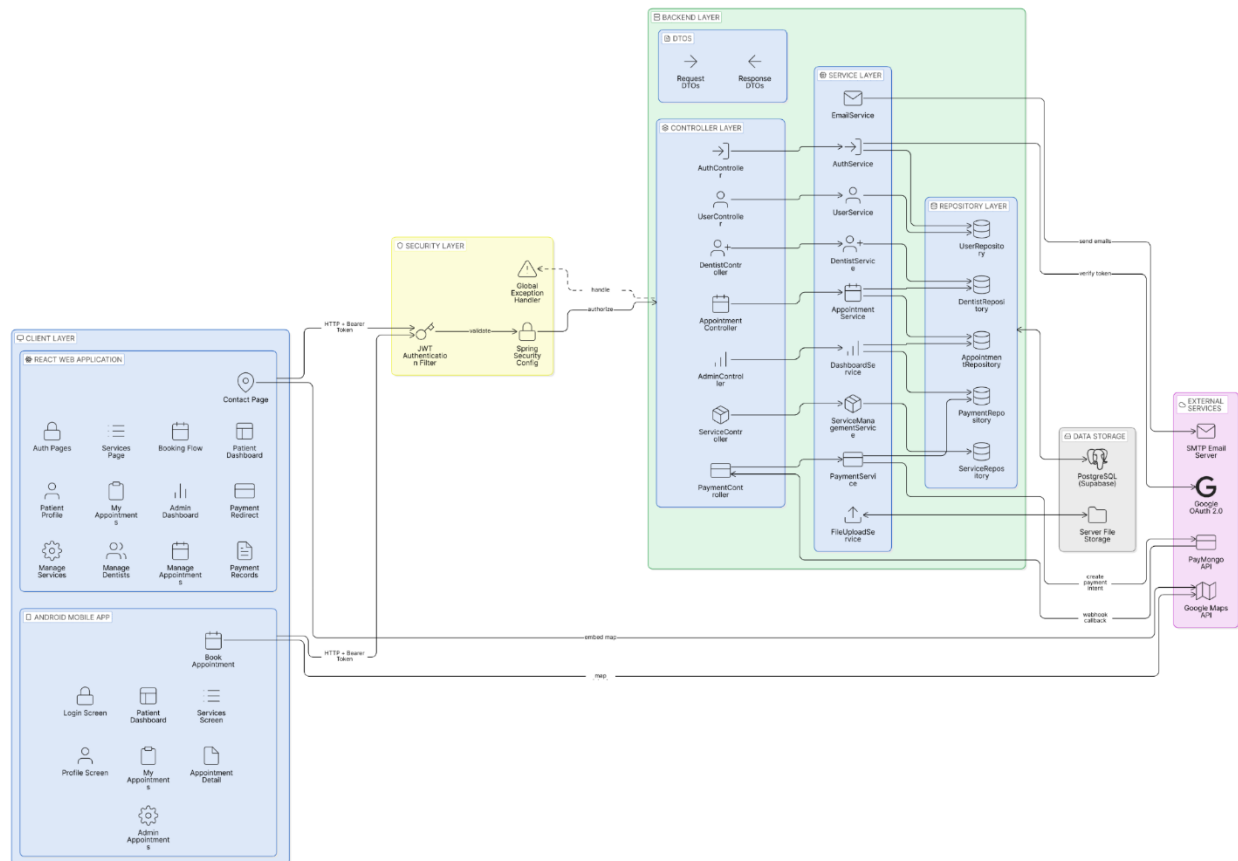
DentaLink follows a Layered Architecture (also known as N-Tier Architecture) on the backend. This separates concerns across distinct layers, making the system modular, testable, and maintainable.

Layer	Responsibility
Controller Layer	Handles HTTP requests and responses. Validates inputs. Routes to the Service layer.
Service Layer	Contains business logic. Calls the Repository layer. Coordinates integrations.
Repository Layer	Data access via Spring Data JPA. Database queries abstracted from service logic.
DTOs	Data Transfer Objects used to structure request/response payloads. Password never exposed.
Security Configuration	Spring Security + JWT filter chain. Role-based endpoint protection.
Global Exception Handler	@RestControllerAdvice — catches and returns structured error responses.
Scheduled Task (Background Job)	@Scheduled component (AppointmentExpiryScheduler) that runs every 5 minutes to auto-cancel unpaid, expired appointments, releasing their slots.

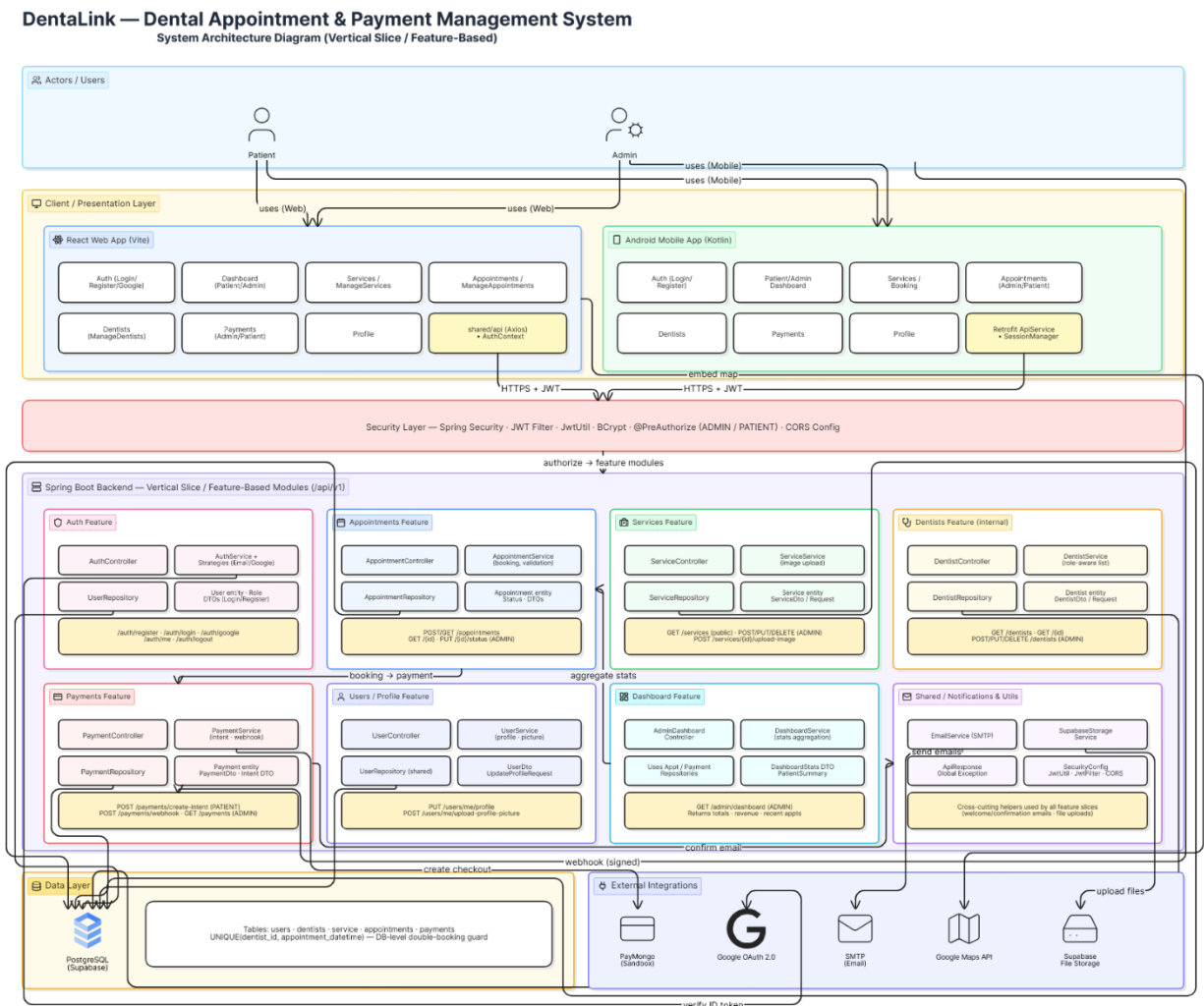
4.2 Technology Stack

Component	Technology
Backend	Java 17+, Spring Boot 3.x, Spring Security, Spring Data JPA, Maven
Database	PostgreSQL 14+ via Supabase Free Tier
Web Frontend	React 18, JavaScript, Axios, React Router
Mobile App	Android Kotlin, XML Layouts, Retrofit, Android API Level 34
Authentication	JWT (JSON Web Token), BCrypt, Google OAuth 2.0
Payment Gateway	PayMongo (Sandbox Mode) — Checkout Sessions API
Email Service	JavaMailSender (SMTP) — e.g., Gmail SMTP
Maps Integration	Google Maps JavaScript API (Web), embedded WebView + Google Maps Intent (Android)
File Storage	Supabase Storage (cloud object storage); public URL saved in the database
Backend Deployment	Render (cloud platform)
Web Deployment	Vercel
Mobile Distribution	APK file

4.3 Component Diagram



4.4 System Architecture Diagram



5.0 API Contract & Communication

5.1 API Standards

Base URL	https://[server-hostname]/api/v1
Format	JSON for all request and response bodies
Authentication	Bearer token (JWT) in Authorization header: Authorization: Bearer <token>
Content-Type	application/json (multipart/form-data for file uploads)
Response Structure	{ "success": boolean, "data": object null, "error": { "code": string, "message": string, "details": object null }, "timestamp": string }

Note: Most endpoints return the full response envelope above. The appointment endpoints (§5.3) currently return a reduced { "success": boolean, "data": object } shape without the error and timestamp fields.

5.2 HTTP Status Codes

Code	Meaning
200 OK	Successful GET or update request
201 Created	Resource successfully created
400 Bad Request	Invalid input, validation failure, or business-rule violation
401 Unauthorized	Authentication required or token invalid/expired
403 Forbidden	Authenticated but insufficient role permissions
404 Not Found	Requested resource does not exist
409 Conflict	Duplicate resource or double-booking conflict
500 Internal Server Error	Unhandled server-side error

5.3 Endpoint Specifications

Authentication Endpoints

POST /auth/register — Patient Registration

Description	Register a new patient account (a welcome email is sent on success)
HTTP Method	POST
Auth Required	None
Request Body	{ "firstName": string, "lastName": string, "email": string, "password": string }
Success Response (201)	{ "success": true, "data": { "user": { "id", "firstName", "lastName", "email", "role", "profileImageUrl" }, "accessToken": string } }
Error Codes	400 — Validation failed 409 — Email already registered

POST /auth/login — Email/Password Login

Description	Authenticate a user and receive a JWT
HTTP Method	POST
Auth Required	None
Request Body	{ "email": string, "password": string }
Success Response (200)	{ "success": true, "data": { "user": { "id", "firstName", "lastName", "email", "role", "profileImageUrl" }, "accessToken": string } }

Error Codes	400 — Missing fields 401 — Invalid credentials
--------------------	--

POST /auth/google — Google OAuth Login

Description	Exchange a Google OAuth ID token for a system JWT (matches an existing user by email or creates a new PATIENT)
HTTP Method	POST
Auth Required	None
Request Body	{ "idToken": string }
Success Response (200)	{ "success": true, "data": { "user": { "id", "firstName", "lastName", "email", "role", "profileImageUrl" }, "accessToken": string } }
Error Codes	400 — Invalid Google token 500 — OAuth verification failed

GET /auth/me — Get Current Authenticated User

Description	Return the profile of the currently authenticated user
HTTP Method	GET
Auth Required	Bearer JWT (any role)
Authorization Header	Authorization: Bearer <token>
Success Response (200)	{ "success": true, "data": { "id", "firstName", "lastName", "email", "role", "profileImageUrl" } }
Error Codes	401 — Token missing or invalid

POST /auth/logout — Logout

Description	JWT tokens are stateless; logout is handled client-side by deleting the token. Token blacklisting is outside the scope of this project.
HTTP Method	POST
Auth Required	None (public endpoint)
Success Response (200)	{ "success": true, "data": "Logged out successfully" }

Service Endpoints

GET /services — List All Services (Public)

Description	Return all dental services
HTTP Method	GET
Auth Required	None (public)
Success Response (200)	{ "success": true, "data": [{ "id", "name", "description", "price", "imageUrl" }] }

GET /services/{id} — Get Single Service (Public)

Description	Return a single dental service by ID
HTTP Method	GET
Auth Required	None (public)
Success Response (200)	{ "success": true, "data": { "id", "name", "description", "price", "imageUrl" } }
Error Codes	404 — Service not found

POST /services — Create Service (Admin Only)

Description	Create a new dental service (image is added separately via the upload-image endpoint)
HTTP Method	POST
Auth Required	Bearer JWT — ADMIN role required
Request Body	{ "name": string, "description": string, "price": number }
Success Response (201)	{ "success": true, "data": { "id", "name", "description", "price", "imageUrl" } }
Error Codes	400 — Validation failed 403 — Not admin

PUT /services/{id} — Update Service (Admin Only)

Description	Update an existing dental service
HTTP Method	PUT
Auth Required	Bearer JWT — ADMIN role required
Path Variable	id — Service ID
Request Body	{ "name": string, "description": string, "price": number }
Success Response (200)	{ "success": true, "data": { updated service object } }
Error Codes	400 403 404

DELETE /services/{id} — Delete Service (Admin Only)

Description	Delete a dental service (blocked if existing appointments reference it)
HTTP Method	DELETE
Auth Required	Bearer JWT — ADMIN role required
Path Variable	id — Service ID
Success Response (200)	{ "success": true, "data": "Service deleted" }
Error Codes	400 — Service has existing appointments 403 404

POST /services/{id}/upload-image — Upload Service Image (Admin Only)

Description	Upload an image for a dental service (stored in Supabase Storage)
HTTP Method	POST
Auth Required	Bearer JWT — ADMIN role required
Content-Type	multipart/form-data
Form Field	file — JPEG or PNG image file (max 5 MB)
Success Response (200)	{ "success": true, "data": { "imageUrl": string } }
Error Codes	400 — Invalid file type or size 403 404

Dentist Endpoints

GET /dentists — List Dentists

Description	Return dentist records. ADMIN receives all dentists; PATIENT receives only ACTIVE dentists.
HTTP Method	GET
Auth Required	Bearer JWT (any role)
Success Response (200)	{ "success": true, "data": [{ "id", "name", "specialization", "status" }] }

GET /dentists/{id} — Get Single Dentist

Description	Return a single dentist record by ID
HTTP Method	GET
Auth Required	Bearer JWT (any role)
Path Variable	id — Dentist ID
Success Response (200)	{ "success": true, "data": { "id", "name", "specialization", "status" } }
Error Codes	401 — Unauthorized 404 — Dentist not found

POST /dentists — Add Dentist (Admin Only)

Description	Create a new dentist record
HTTP Method	POST
Auth Required	Bearer JWT — ADMIN role required
Request Body	{ "name": string, "specialization": string, "status": "ACTIVE" "INACTIVE" }
Success Response (201)	{ "success": true, "data": { "id", "name", "specialization", "status" } }

Error Codes	400 — Invalid status 403
--------------------	----------------------------

PUT /dentists/{id} — Update Dentist (Admin Only)

Description	Update a dentist record
HTTP Method	PUT
Auth Required	Bearer JWT — ADMIN role required
Path Variable	id — Dentist ID
Request Body	{ "name": string, "specialization": string, "status": string }
Success Response (200)	{ "success": true, "data": { updated dentist object } }
Error Codes	400 403 404

DELETE /dentists/{id} — Delete Dentist (Admin Only)

Description	Delete a dentist record (blocked if existing appointments reference the dentist — set status to INACTIVE instead)
HTTP Method	DELETE
Auth Required	Bearer JWT — ADMIN role required
Path Variable	id — Dentist ID
Success Response (200)	{ "success": true, "data": "Dentist deleted" }
Error Codes	400 — Dentist has existing appointments 403 404

Appointment Endpoints

POST /appointments — Create Appointment (Patient Only)

Description	Book a new appointment. Validates future date, no-Sunday, clinic operating hours, active dentist, and double-booking before creation.
HTTP Method	POST
Auth Required	Bearer JWT — PATIENT role required
Request Body	{ "serviceId": long, "dentistId": long, "appointmentDatetime": "YYYY-MM-DDTHH:MM:SS" }
Success Response (201)	{ "success": true, "data": { "id", "serviceId", "dentistId", "appointmentDatetime", "status": "PENDING_PAYMENT", "paymentStatus": "UNPAID", ...service/dentist/patient details } }
Error Codes	400 — Missing fields or business-rule violation (past date, Sunday, outside operating hours, inactive dentist) 403 — Not a patient 409 — Dentist already booked at this time

GET /appointments — List Appointments

Description	Returns appointments based on role. Admin receives all appointments; Patient receives only their own. Role is determined from the JWT token.
HTTP Method	GET
Auth Required	Bearer JWT (ADMIN or PATIENT)
Query Params (Admin)	status — filter by appointment status (optional, Admin only)
Success Response (200)	{ "success": true, "data": [{ appointment objects }] }
Error Codes	401 — Token missing or invalid 403 — Insufficient role

GET /appointments/{id} — Get Appointment Detail

Description	Get details of a specific appointment
HTTP Method	GET
Auth Required	Bearer JWT — must be owner (PATIENT) or ADMIN
Path Variable	id — Appointment ID
Success Response (200)	{ "success": true, "data": { "id", "serviceId", "dentistId", "serviceName", "servicePrice", "serviceImageUrl", "dentistName", "dentistSpecialization", "patient": { "id", "firstName", "lastName", "email" }, "appointmentDatetime", "status", "paymentStatus" } }
Error Codes	403 404

POST /appointments/{id}/cancel — Cancel Own Appointment (Patient Only)

Description	Allows a patient to cancel their own appointment while it is still unpaid (PENDING_PAYMENT). Cancellation after payment is not permitted.
HTTP Method	POST
Auth Required	Bearer JWT — PATIENT role required
Path Variable	id — Appointment ID
Success Response (200)	{ "success": true, "data": { updated appointment object with status = CANCELLED } }
Error Codes	400 — Appointment already paid 403 — Not the owner 404

PUT /appointments/{id}/status — Update Appointment Status (Admin Only)

Description	Update appointment status to COMPLETED or CANCELLED
HTTP Method	PUT
Auth Required	Bearer JWT — ADMIN role required

Path Variable	id — Appointment ID
Request Body	{ "status": "COMPLETED" "CANCELLED" }
Success Response (200)	{ "success": true, "data": { updated appointment object } }
Error Codes	400 — Invalid status, or COMPLETED requested on an unpaid appointment 403 404
Restrictions	Admin cannot set status to CONFIRMED (only the PayMongo webhook can). COMPLETED is allowed only when payment_status = PAID.

Payment Endpoints

POST /payments/create-intent — Create PayMongo Checkout Session

Description	Create a PayMongo Checkout Session for the given appointment and return the hosted checkout URL
HTTP Method	POST
Auth Required	Bearer JWT — PATIENT role required
Request Body	{ "appointmentId": long }
Success Response (200)	{ "success": true, "data": { "checkoutUrl": string, "paymentIntentId": string } }
Note	paymentIntentId holds the PayMongo Checkout Session ID (cs_...), which is stored on the appointment for webhook correlation.
Error Codes	400 — Appointment not found, not owned, or already paid 403

POST /payments/webhook — PayMongo Webhook Handler

Description	Receive PayMongo webhook events on payment success
HTTP Method	POST
Auth Required	None (PayMongo webhook signature verified via HMAC-SHA256 inside the service layer)
Trigger	Automatically called by PayMongo. Only the checkout_session.payment.paid event is processed; other events (including payment.paid) are silently acknowledged to avoid duplicate processing.
Action	Verify signature → correlate by checkout session ID → update payment_status = PAID → update appointment status = CONFIRMED → send confirmation email
Behavior	Always returns 200 (idempotent). Invalid signatures, unknown appointments, cancelled appointments, and already-paid appointments are acknowledged without state change.

Success Response (200)	<code>{ "success": true, "data": "Webhook received" }</code>
-------------------------------	--

GET /payments — List All Payment Records (Admin Only)

Description	Return all payment records for admin monitoring
HTTP Method	GET
Auth Required	Bearer JWT — ADMIN role required
Success Response (200)	<code>{ "success": true, "data": [{ "id", "appointmentId", "serviceName", "paymongoPaymentId", "paymentAmount", "paymentStatus", "paymentCreatedAt", "patient": { "id", "firstName", "lastName" } }] }</code>
Error Codes	403 — Not admin

User Profile Endpoints

PUT /users/me/profile — Update Patient Profile

Description	Update first name, last name, or change password. Email is not updatable. currentPassword is required for email/password accounts.
HTTP Method	PUT
Auth Required	Bearer JWT (PATIENT)
Request Body	<code>{ "firstName": string, "lastName": string, "currentPassword": string, "newPassword": string (optional, min 8 chars) }</code>
Success Response (200)	<code>{ "success": true, "data": { updated user profile (no password) } }</code>
Error Codes	400 — Missing current password or weak new password 401 — Current password incorrect

POST /users/me/upload-profile-picture — Upload Profile Picture

Description	Upload a patient profile picture (stored in Supabase Storage)
HTTP Method	POST
Auth Required	Bearer JWT (PATIENT)
Content-Type	multipart/form-data
Form Field	file — JPEG or PNG image (max 5 MB)
Success Response (200)	<code>{ "success": true, "data": { "profileImageUrl": string } }</code>
Error Codes	400 — Invalid file type or size

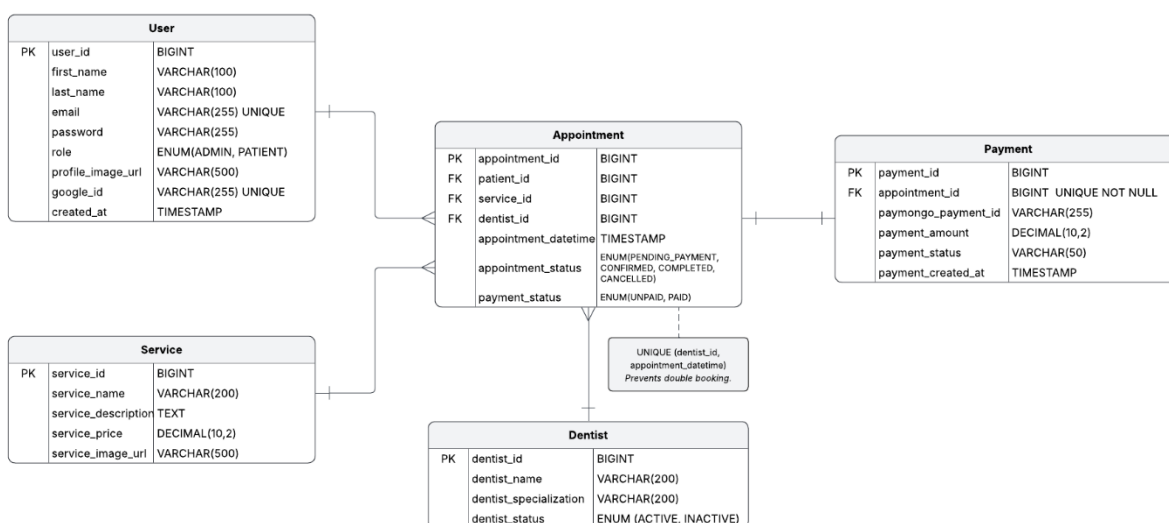
Admin Dashboard Endpoint

GET /admin/dashboard — Dashboard Summary (Admin Only)

Description	Return statistics for the admin dashboard
HTTP Method	GET
Auth Required	Bearer JWT — ADMIN role required
Success Response (200)	<pre>{ "success": true, "data": { "totalAppointments": int, "pendingPayments": int, "confirmedAppointments": int, "completedAppointments": int, "cancelledAppointments": int, "pendingPaymentAppointments": int, "totalRevenue": decimal, "recentAppointments": [{ "id", "patientName", "serviceName", "dentistName", "appointmentDatetime", "appointmentStatus" }] }</pre>

6.0 Database Design

6.1 Entity Relationship Diagram



Note: Double-booking prevention is enforced at the application/service layer — the system rejects a new appointment when a non-cancelled appointment already exists for the same `dentist_id` and `appointment_datetime` (409 Conflict). Slots whose appointment was cancelled are released and may be rebooked.

Detailed Relationships:

- **User → Appointment (One-to-Many):** A single patient user account can be associated with zero or many appointment records over time. Each appointment must belong to exactly one patient, enforced via the `patient_id` foreign key. Deleting a user does not automatically delete appointments.
- **Service → Appointment (One-to-Many):** A single dental service can be referenced by zero or many appointments. Each appointment references exactly one service via the `service_id` foreign key.

- **Dentist → Appointment (One-to-Many):** A single dentist record can be assigned to zero or many appointments across different dates and times. Each appointment has exactly one assigned dentist via the `dentist_id` foreign key. The application prevents the same dentist from being double-booked at the same date and time.
- **Appointment → Payment (One-to-One):** Each appointment has at most one corresponding payment record, enforced via a unique foreign key on `payments.appointment_id`. No appointment can be paid twice, and no payment can exist without a corresponding appointment.

Key Tables:

1. **User** — Stores all system user accounts. Holds credentials (BCrypt-hashed password, nullable for OAuth users), role assignment (ADMIN or PATIENT), the Google OAuth identifier for social-login users, and the profile image URL.
2. **Dentist** — Stores internal dentist records managed exclusively by the admin. Dentists do not have login accounts. Used for appointment assignment and availability tracking via the ACTIVE/INACTIVE status field.
3. **Service** — Stores the dental service catalog. Each service has a name, description, price, and an optional image URL set by the admin via file upload. Services are publicly readable but only admin-modifiable.
4. **Appointment** — The central transactional table. Links patients, services, and dentists in a single booking record. Tracks both the lifecycle status (PENDING_PAYMENT → CONFIRMED → COMPLETED / CANCELLED) and the payment status (UNPAID → PAID). Stores the PayMongo checkout session ID (`paymongo_intent_id`) used for webhook correlation, and a `created_at` timestamp used by the auto-expiry scheduler.
5. **Payment** — Stores payment records created and updated exclusively via the PayMongo webhook. Holds the PayMongo payment ID and the checkout session ID for audit reference, the amount charged, and the payment status.

Table Structure Summary

users: `user_id` (PK) · `first_name` · `last_name` · `email` (UNIQUE) · `password` (BCrypt, nullable for OAuth users) · `role` ENUM(ADMIN, PATIENT) · `profile_image_url` · `google_id` (nullable) · `created_at` TIMESTAMP

dentists: `dentist_id` (PK) · `dentist_name` · `dentist_specialization` · `dentist_status` ENUM(ACTIVE, INACTIVE)

services: `service_id` (PK) · `service_name` · `service_description` · `service_price` DECIMAL(10,2) · `service_image_url`

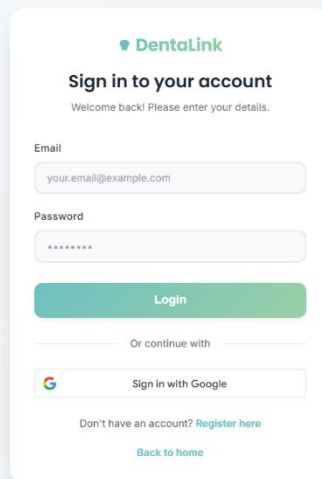
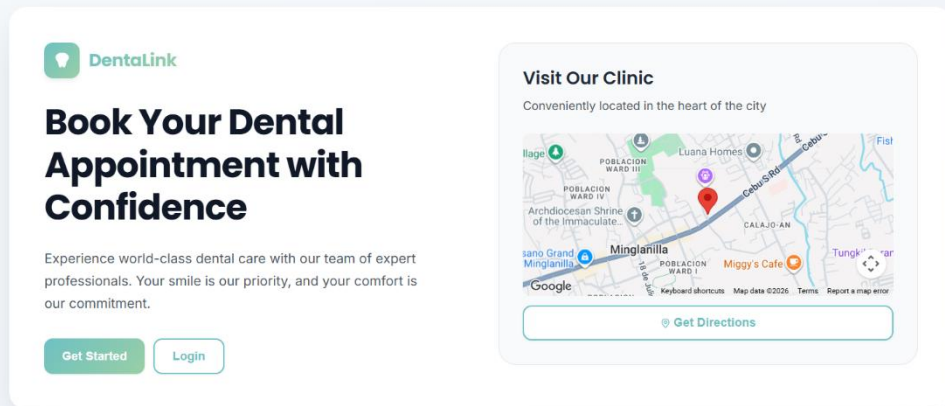
appointments: `appointment_id` (PK) · `patient_id` (FK → `users.user_id`) · `service_id` (FK → `services.service_id`) · `dentist_id` (FK → `dentists.dentist_id`) · `appointment_datetime` · `appointment_status` ENUM(PENDING_PAYMENT, CONFIRMED, COMPLETED, CANCELLED) · `payment_status` ENUM(UNPAID, PAID) · `paymongo_intent_id` · `created_at` TIMESTAMP

payments: `payment_id` (PK) · `appointment_id` (FK → `appointments.appointment_id`, UNIQUE) · `paymongo_payment_id` · `paymongo_intent_id` · `payment_amount` DECIMAL(10,2) · `payment_status` · `payment_created_at` TIMESTAMP

7.0 UI/UX Design

Figma Link: figma.com/design/viAbWUakKg91f3RgO481AT/DentaLink?node-id=0-1&p=f&t=eMGrLD7U1bOfCMqG-0

7.1 Web Application Wireframes





Create your account

Join us for better dental care

First Name

Last Name

John

Smith

Email

your.email@example.com

Password

Confirm Password

Register

Or sign up with



Sign up with Google

Already have an account? [Login here](#)

[Back to home](#)

Patient/User



DashboardServicesMy AppointmentsContact

Welcome back, Frances Anne!

Good afternoon

Frances Anne Riconalla

Ready for your dental visit?

Book Appointment

Upcoming Appointments

0

Completed Visits

0

Pending Payments

0

Upcoming Appointments

View All

No upcoming appointments

Book a service to get started

Recent History

View All

No history yet

Your completed visits will appear here

Our Services

Choose from our comprehensive range of dental services



Dental Checkup

Test Test Test

₱500.00

Book Appointment



Cleaning

Lorem Ipsum

₱700.00

Book Appointment



Whitening

Lorem Ipsum

₱1,000.00

Book Appointment



Extraction

Lorem Ipsum

₱500.00

Book Appointment

Our Services

Choose from our comprehensive range of dental services



Dental Checkup

Test Test Test

₱500.00

Book Appointment

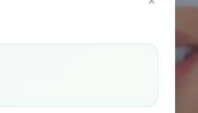


Cleaning

Lorem Ipsum

₱700.00

Book Appointment



Whitening

Lorem Ipsum

₱1,000.00

Book Appointment



Extraction

Lorem Ipsum

₱500.00

Book Appointment

Book Appointment

Dental Checkup

TOTAL AMOUNT

₱500.00

Select Dentist

Choose a dentist...

Preferred Date

dd/mm/yyyy

Preferred Time

Choose a time...

Your appointment will be created with a Pending Payment status. After booking, you can proceed with payment.

Cancel

Book & Pay



Payment Confirmed!

Your appointment has been confirmed. See you soon!

APPOINTMENT DETAILS

Service	Dental Checkup
Dentist	Dr. Dr. Vaughn Moralde
Date	Saturday, May 23, 2026
Time	3:00 PM
Status	Confirmed

View My Appointments

My Appointments

View and track your dental appointments

- All
- Confirmed
- Pending Payment
- Completed
- Cancelled

SERVICE	DENTIST	DATE	TIME	STATUS	PAYMENT	ACTION
Dental Checkup	Dr. Dr. Vaughn Moralde	May 23, 2026	3:00 PM	Confirmed	Paid	

Contact Us

Find us, get directions, or reach out to our team

Get in Touch

ADDRESS
3rd Floor, One Cybersquare
Cebu IT Park, Archbishop Reyes Ave
Apas, Cebu City, 6000

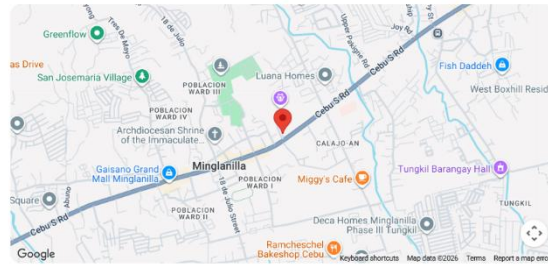
PHONE
(032) 234-5678
+63 917 234 5678 Globe

EMAIL
hello@dentalink.ph

SOCIAL MEDIA
DentalLink Dental Clinic
@dentalink_clinic

Clinic Hours

Monday – Friday 8:00 AM – 6:00 PM
Saturday 9:00 AM – 5:00 PM
Sunday **Closed**



DentalLink Dental Clinic
Cebu IT Park, Cebu City, Philippines

[Get Directions](#)

FR Frances Anne ▾

Frances Anne Riconalla

riconallafrancesanne@gmail.com

 My Profile

 Log Out

My Profile

Manage your account information

Profile Picture



Change Photo

JPEG or PNG - max 5 MB

Personal Information

Email

riconallafrancesanne@gmail.com

Email cannot be changed.

First Name

Frances Anne

Last Name

Riconalla

Change Password (optional)

Current Password *

Required to save any changes

New Password (leave blank to keep current)

Minimum 8 characters

Save Changes

Admin

Dashboard

Friday, May 22, 2026

Total Appointments
2Pending Payments
0Confirmed (Total)
1Total Revenue
P1,000.00

Appointment Status



Pending Payment	0	0%
Confirmed	1	50%
Completed	1	50%
Cancelled	0	0%

Recent Appointments

[View All](#)

PATIENT	SERVICE	DENTIST	DATE & TIME	STATUS
Frances Anne Riconalla	Dental Checkup	Dr. Vaughn Moralde	May 23, 2026 - 3:00 PM	Confirmed
Florence Anne Riconalla	Dental Checkup	Dr. Vaughn Moralde	May 20, 2026 - 1:30 PM	Completed

Manage Services

Create and manage dental service offerings

[+ Add New Service](#)

Dental Checkup

Test Test Test

₱500.00

[Edit](#)[Delete](#)

Cleaning

Lorem Ipsum

₱700.00

[Edit](#)[Delete](#)

Whitening

Lorem Ipsum

₱1,000.00

[Edit](#)[Delete](#)

Extraction

Lorem Ipsum

₱500.00

[Edit](#)[Delete](#)

Manage Services

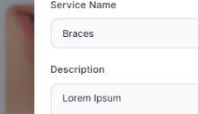
Create and manage dental service offerings

[+ Add New Service](#)

Dental Checkup

Test Test Test

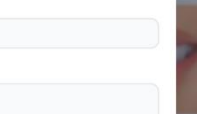
₱500.00

[Edit](#)[Delete](#)

Cleaning

Lorem Ipsum

₱700.00

[Edit](#)[Delete](#)

Whitening

Lorem Ipsum

₱1,000.00

[Edit](#)[Delete](#)

Extraction

Lorem Ipsum

₱500.00

[Edit](#)[Delete](#)

Add New Service

Service Name

Braces

Description

Lorem Ipsum

Price (₱)

498

Service Image

[Choose File](#) what-to-expect-dental-checkup.jpg

[Cancel](#)[Save Service](#)

Manage Dentists

Manage dentist records and availability

+ Add Dentist

All Active Inactive

#	NAME	SPECIALIZATION	STATUS	ACTIONS
1	 Dr. Vaughn Moralde	Orthodontics	ACTIVE	 

Manage Appointments

View and update all patient appointments

All Status Pending Payment Confirmed Completed Cancelled

PATIENT	SERVICE	DENTIST	DATE & TIME	STATUS	PAYMENT	ACTION
Florence Anne Riconalia riconaliaflorenceanne@gmail.com	Dental Checkup	Dr. Vaughn Moralde	May 20, 2026 - 1:30 PM	Completed	Paid	—
Frances Anne Riconalia riconaliafrancesanne@gmail.com	Dental Checkup	Dr. Vaughn Moralde	May 23, 2026 - 3:00 PM	Confirmed	Paid	Update Status ▾

Payment Records

Records are updated automatically via PayMongo webhook. No manual edits permitted.

Total Payments

2

Paid

2

Total Revenue

₱1,000.00

PAYMENT ID	APPOINTMENT	PATIENT	AMOUNT	PAYMONGO REF	STATUS	DATE
#15	Dental Checkup Appt #36	Florence Anne Riconalla	₱500.00	pay_5mshulgl82C313PKgi-FkQ4u5uA	Paid	May 18, 2026
#16	Dental Checkup Appt #37	Frances Anne Riconalla	₱500.00	pay_nxagTD7yomeFaLPh60F4Hxjc	Paid	May 22, 2026

7.2 Mobile Application Wireframes

The image displays two mobile application wireframes for 'DentaLink'. Both screens feature a teal header bar with status icons and a light blue background.

Left Wireframe (9:50:12):

- Header: DentaLink logo (teal tooth icon) and 'Create your account' text.
- Form Fields (all empty):
 - First Name
 - Last Name
 - Email
 - Password
 - Confirm Password
- Bottom Button: 'Create Account' (teal).

Right Wireframe (10:04:15):

- Header: DentaLink logo and 'Create your account' text.
- Form Fields (filled with sample data):
 - First Name: Florence
 - Last Name: Riconolla
 - Email: riconollaflorenceanne@gmail.com
 - Password:
 - Confirm Password:
- Bottom Button: 'Create Account' (teal).
- Footer: 'Already have an account? Login' (teal text).

Both wireframes include a black Android navigation bar at the bottom.



DentaLink

Sign in to your account

Email

you@example.com

Password

Your password

Sign In

or continue with



Sign in with Google

Don't have an account? [Register](#)

Patient/User

DentaLink

Welcome, Frances Anne!

Your dental appointment overview

1

Upcoming

0

Completed

1

Pending Pay

Book Appointment

Get Clinic Directions

Upcoming Appointments

View All →

Veneers

PENDING PAYMENT

Dr. Vaughn Moralde

May 29, 2026 8:00 AM

Payment: UNPAID

←

Our Services

Choose a service to book your appointment



Tooth Extraction

Safe and painless removal of damaged, decayed, or problematic teeth.

₱800.00

Book Now



Teeth Cleaning

Professional dental cleaning to remove plaque, tartar, and stains for healthier te...

₱500.00

Book Now



Teeth Whitening

Cosmetic whitening treatment to brighten and enhance your smile safely.

₱3,500.00

Book Now



Veneers

Thin porcelain shells placed on teeth to improve appearance and smile aesthetics.

₱15,000.00

Book Now



Test Service

← Book Appointment

Test Service

₱500.00

DENTIST

Vaughn Moralde - Orthodontics

DATE

May 28, 2026



TIME SLOT

08:00

Book and Pay

Cancel



DentaLink

Welcome, Frances Anne!

Dashboard

Services

My Appointments



Profile

Clinic Directions

Log Out

← My Appointments

All

Confirmed

Pending Payment

Co

Tooth Extraction

CANCELLED

Dr. Vaughn Moralde

May 30, 2026 9:00 AM

Payment: UNPAID

Test Service

CONFIRMED

Dr. Vaughn Moralde

May 28, 2026 10:30 AM

Payment: PAID

Teeth Cleaning

CANCELLED

Dr. Vaughn Moralde

Jun 5, 2026 3:30 PM

Payment: PAID

Veneers

PENDING PAYMENT

Dr. Vaughn Moralde

May 29, 2026 8:00 AM

Payment: UNPAID

Cancel

Tap to Pay

← My Profile

Change Photo

ACCOUNT INFORMATION

Email

riconallafrancesanne@gmail.com

Email cannot be changed

Frances Anne

Riconalla

CHANGE PASSWORD

Current Password (required to save)

← Clinic Directions



DentaLink Dental Clinic

Minglanilla, Cebu, Philippines

Open in Google Maps

Admin

≡ Admin Dashboard

Admin Dashboard

Manage your dental clinic

5

Total Appts

2

Pending Pay

1

Confirmed

Total Revenue

₱1,800.00

Quick Actions

Services

Dentists

Appointments

Payments

Recent Appointments

Frances Anne Riconalla

CONFIRMED

Dr. Vaughn Moralde

May 28, 2026 10:30 AM

Frances Anne Riconalla

PENDING PAYMENT

← Manage Services

+ Add



Tooth Extraction

Safe and painless removal of damaged, decayed, or problematic teeth.

₱800.00

Edit



Teeth Cleaning

Professional dental cleaning to remove plaque, tartar, and stains for healthier te...

₱500.00

Edit



Teeth Whitening

Cosmetic whitening treatment to brighten and enhance your smile safely.

₱3,500.00

Edit



Veneers

Thin porcelain shells placed on teeth to improve appearance and smile aesthetics.

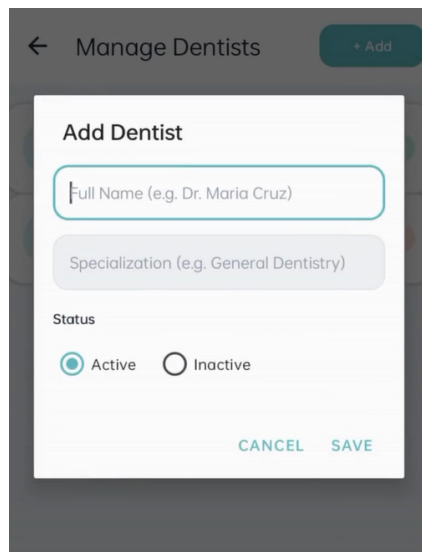
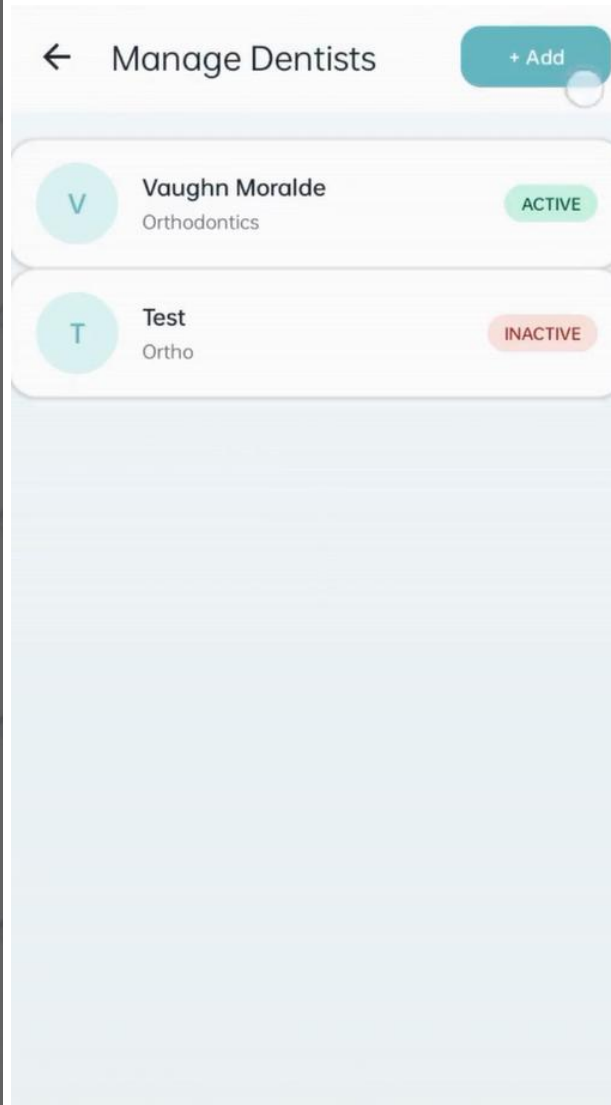
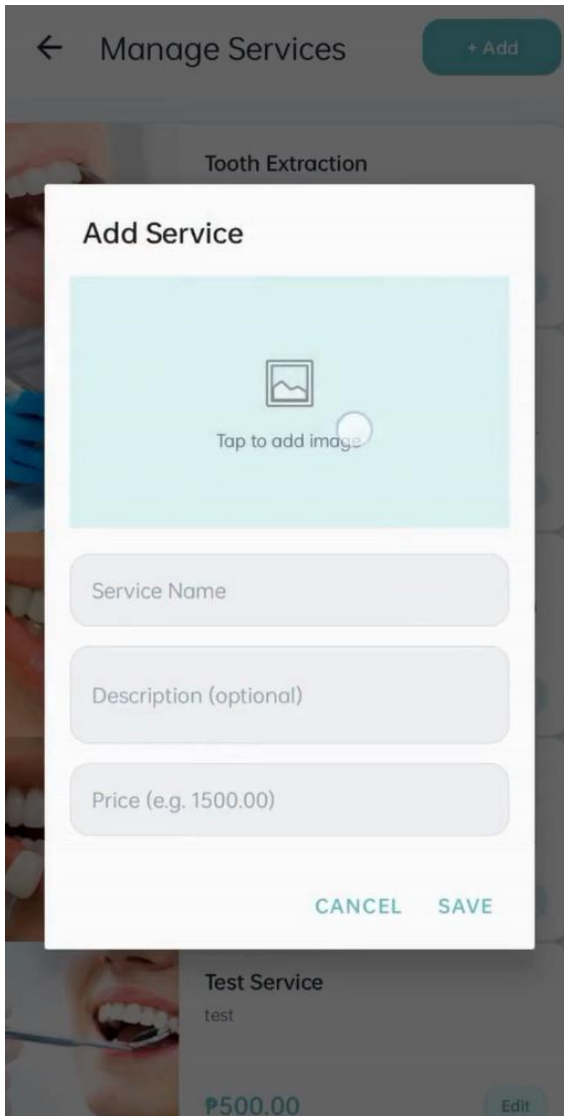
₱15,000.00

Edit



Test Service

test



← Manage Appointments

Filter: All



Tap card to update

Test Service

CONFIRMED

Patient: Frances Anne Riconalla

Dr. Vaughn Moralde

May 28, 2026 10:30 AM



Payment

Tap to update >

PAID

Tooth Extraction

COMPLETED

Patient: Florence Anne

Dr. Vaughn Moralde

May 28, 2026 2:30 PM

Payment

PAID

Teeth Cleaning

CANCELLED

Patient: Frances Anne Riconalla

Dr. Vaughn Moralde

Jun 5, 2026 3:30 PM

Payment

PAID

Veneers

PENDING PAYMENT

Patient: Frances Anne Riconalla

Dr. Vaughn Moralde

May 29, 2026 8:00 AM

← Payment Records

Records are updated automatically via PayMongo webhook

Appt #71

PAID

Florence Anne

Tooth Extraction

₱800.00

Ref: pay_XKH7eVVXwht3XNKMi5YfQsps

Appt #73

PAID

Frances Anne Riconalla

Teeth Cleaning

₱500.00

Ref: payJM5N8PPqnfWK6Ukk7NDQGMKr

Appt #75

PAID

Frances Anne Riconalla

Test Service

₱500.00

Ref: pay_VUMWsMGWg3nHm6YpEgjQc3LN

8.0 Plan

8.1 Project Timeline

Phase	Activities	Duration
Phase 1: Planning & Design	Requirements finalization, SDD completion, architecture design, API specification, database design, UI wireframes	Week 1–2
Phase 2: Backend Development	Spring Boot setup, JWT + BCrypt auth, user registration/login, Google OAuth, service/dentist CRUD, appointment booking + validation, auto-expiry scheduler, PayMongo integration, webhook handler, SMTP email, Supabase file upload	Week 3–5
Phase 3: Web Frontend	React setup, auth pages, protected routes, patient dashboard, services page, booking flow, payment redirect, My Appointments, admin dashboard, admin management pages	Week 5–7
Phase 4: Mobile App	Android project setup, Retrofit + JWT, login screen, services screen, booking screen, appointments screen, profile screen, admin status update, protected endpoints	Week 7–9
Phase 5: Integration & Testing	End-to-end testing, bug fixes, security review, PayMongo webhook testing, Google Maps verification, SMTP testing	Week 9–10
Phase 6: Deployment	Backend → Render, Web → Vercel, Mobile → APK, final documentation	Week 10

8.2 Milestones

Milestone	Deliverable	Target Date	Status
M1	All design documents complete (SDD finalized)	End of Week 2	Complete
M2	Backend API fully functional (auth, core modules, integrations)	End of Week 5	Complete
M3	Web application complete (all pages and flows working)	End of Week 7	Complete
M4	Mobile application complete (all screens + protected endpoints)	End of Week 9	Complete
M5	Full system deployed and integrated — defense-ready	End of Week 10	Complete

8.3 Risk Mitigation

Risk	Impact	Mitigation
PayMongo webhook not receiving locally	Payment flow cannot be tested	Use an ngrok tunnel for local webhook testing during development
Google OAuth token issues	OAuth login blocked	Implement email/password login as fallback; test OAuth early
Supabase free tier limitations	DB connections throttled	Optimize queries; use connection pooling; test with realistic data
Android Kotlin XML layout complexity	UI development slow	Use standard Material Design components; avoid custom views initially
Time constraints on all platforms	Incomplete features	Build core flows first; defer COULD HAVE features to last week

8.4 Repository Structure

Repository name: IT342_DentaLink_G2_Riconalla

Folder	Contents
backend/	Spring Boot project (Maven) — all Java source, config, resources
web/	React application — all components, pages, services, config
mobile/	Android Studio project (Kotlin + XML) — all source and layouts
docs/	SDD, ERD, architecture diagrams, wireframes
README.md	Project overview, setup instructions, team info